

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Информационная безопасность

Работа №1

«Разработка защищенного REST API с интеграцией в CI/CD»

Болорболд Аригуун

Группа: P3411

Выполнение

Ссылка на публичный git-репозиторий:

<https://github.com/17starplatinum/MagicItems>

Файл README.md в репозиторий:

Информационная безопасность. Лабораторная работа 1

Описание проекта

Этот проект демонстрирует создание безопасного backend-приложения с помощью Java + Spring Boot, который включает:

- аутентификацию пользователей
 - управление постами
 - защиту от таких атак, как SQL-инъекции, XSS-атаки, Broken Authentication
-

Методы API

1. POST /auth/login — Аутентификация пользователя

Описание: Метод принимает логин и пароль пользователя. В случае успешной аутентификации возвращает JWT-токен.

Метод: POST

URL: <http://localhost:8080/api/v1/auth/login>

Запрос:

POST /auth/login

```
{  
  "username": "{{username}}",  
  "password": "{{password}}"  
}
```

Заголовки:

Content-Type: application/json

Успешный ответ (200 OK):

```
{  
  "token": "eyJhbGciOiJIUzI1NiJ9..."  
}
```

Ошибки:

- 403 Forbidden — если логин/пароль неверны:

- {
- "code": 400,
- "message": "Bad credentials",
- "timestamp": 1760509489173

}

2. GET /api/v1/magic-items — Получение всех волшебных предметов

Описание: Возвращает список сущностей MagicItems. Доступ только для аутентифицированных пользователей (с валидным JWT).

Метод: GET

URL: <http://localhost:8080/api/v1/magic-items>

Запрос:

GET /api/v1/magic-items

Заголовки:

Authorization: Bearer <JWT_TOKEN>

Content-Type: application/json

Успешный ответ (200 OK):

```
[
  {
    "id": "3ca83b3f-cf51-48cd-9871-36ee2b223a47",
    "name": "Rainbow Bayonet",
    "description": "Cutter of reality",
    "rarity": "GODLIKE",
    "powerLevel": 10000,
    "cooldownSeconds": 2,
    "cursed": false,
    "universalNature": "NONE",
    "createdAt": "2025-10-15T06:15:54.002888100Z"
  },
  {
    "id": "...",
    "name": "..."
  }
]
```

Ошибки:

- 403 Forbidden — если отсутствует или невалиден JWT.
- {
- "timestamp": "2025-10-15T06:30:34.412+00:00",
- "status": 403,
- "error": "Forbidden",
- "path": "/api/v1/magic-items"
- }

3. POST /api/v1/magic-items — Добавление нового волшебного предмета

Описание: Добавляет новую сущность MagicItem в список. Доступ только для аутентифицированных пользователей. **Метод:** POST

URL: <http://localhost:8080/api/v1/magic-items>

Запрос:

POST /auth/login

```
{  
  "name": "Rainbow Bayonet",  
  "description": "Cutter of reality",  
  "rarity": "GODLIKE",  
  "powerLevel": 10000,  
  "cooldownSeconds": 2,  
  "cursed": false,  
  "universalNature": "NONE"  
}
```

Заголовки:

Authorization: Bearer <JWT_TOKEN>

Content-Type: application/json

Успешный ответ (200 OK):

```
{  
  "id": "b90db3c0-b9d5-4bc2-ab6b-ca0c91baaaeb",  
  "name": "Rainbow Bayonet",  
  "description": "Cutter of reality",  
  "rarity": "GODLIKE",  
  "powerLevel": 10000,  
  "cooldownSeconds": 2,
```

```
"cursed": false,  
"universalNature": "NONE",  
"createdAt": "2025-10-15T06:35:21.863282500Z"  
}
```

Ошибки:

- 403 Forbidden — если отсутствует или невалиден JWT.
 - {
 - "timestamp": "2025-10-15T06:30:34.412+00:00",
 - "status": 403,
 - "error": "Forbidden",
 - "path": "/api/v1/magic-items"
- ```
}
```
- 400 Bad Request — если не пройдена серверная валидация объекта запроса.
  - {
    - "code": 400,
    - "message": "Could not commit JPA transaction",
    - "timestamp": 1760510244550
- ```
}
```

4. GET /api/v1/magic-items/{id} — Получение волшебного предмета по его ID

Описание: Возвращает сущность MagicItems, соответствующий передаваемому ID. Доступ только для аутентифицированных пользователей (с валидным JWT).

Метод: GET

URL: <http://localhost:8080/api/v1/magic-items/{id}>

Запрос:

GET /api/v1/magic-items/{id}

Заголовки:

Authorization: Bearer <JWT_TOKEN>

Content-Type: application/json

Успешный ответ (200 OK):

```
{  
  "id": "3ca83b3f-cf51-48cd-9871-36ee2b223a47",  
  "name": "Rainbow Bayonet",  
  "description": "Cutter of reality",  
  "rarity": "GODLIKE",  
}
```

```

    "powerLevel": 10000,
    "cooldownSeconds": 2,
    "cursed": false,
    "universalNature": "NONE",
    "createdAt": "2025-10-15T06:15:54.002888100Z"
  }

```

Ошибки:

- 403 Forbidden — если отсутствует или невалиден JWT.
- {
 - "timestamp": "2025-10-15T06:30:34.412+00:00",
 - "status": 403,
 - "error": "Forbidden",
 - "path": "/api/v1/magic-items"
- 404 Not Found — если отсутствует волшебный предмет, соответствующий заданному ID.
- {
 - "status": 404,
 - "error": "Предмета с id = 3ca83b3f-cf51-48cd-9871-36ee2b223ab1 не существует.",
 - "timestamp": 1760509489024

Реализованные меры защиты

API использует **JWT-токены** для защиты эндпоинтов.

- Пользователь проходит аутентификацию через POST /auth/login.
- В ответ получает accessToken (JWT).
- Для доступа к защищённым эндпоинтам необходимо передавать токен в заголовке: Authorization: Bearer <JWT>

Противодействие SQL Injection (SQLi)

Используется Spring Data JDBC, который применяет параметризованные запросы → данные подставляются безопасно, без конкатенации SQL-строк. Таким образом, классические SQLi невозможны.

Реализация:

@Repository

```
public interface MagicItemRepository extends JpaRepository<MagicItem, UUID> {
```

```
}
```

Противодействие XSS

REST API возвращает только JSON и не исполняет JavaScript. Устанавливаются следующие заголовки HTTP:

X-XSS-Protection: 1; mode=block

Content-Security-Policy: script-src 'self'

Реализация:

```
.headers(headers ->
    headers.xssProtection(
        xss ->
xss.headerValue(XXssProtectionHeaderWriter.HeaderValue.ENABLED_MODE_BLOCK)
    ).contentSecurityPolicy(
        cps -> cps.policyDirectives("script-src 'self'")
    ));
```

Реализация аутентификации

- Используется JWT-токен, который подписывается секретным ключом (HMAC SHA-256 подпись).
- Само хэширование реализуется путём использования BCrypt.
- Токен проверяется middleware (Spring Security OncePerRequestFilter).
- Без токена доступ к защищённым эндпоинтам (/api/v1/magic-items) запрещён.

Реализация:

```
http.csrf(AbstractHttpConfigurer::disable)

    .authorizeHttpRequests(auth -> auth
        .requestMatchers("api/v1/auth/login").permitAll()
        .requestMatchers("/api/v1/magic-items").authenticated()
        .anyRequest().permitAll()
    )
```

...

```
http.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
```

Отчёты на SAST/SCA:

Run SpotBugs

```
1 ▶ Run ./gradlew spotbugsMain spotbugsTest
7
7 > Task :compileJava UP-TO-DATE
8 > Task :processResources UP-TO-DATE
9 > Task :classes UP-TO-DATE
10 > Task :spotbugsMain UP-TO-DATE
11 > Task :compileTestJava NO-SOURCE
12 > Task :processTestResources NO-SOURCE
13 > Task :testClasses UP-TO-DATE
14 > Task :spotbugsTest NO-SOURCE
15 BUILD SUCCESSFUL in 1s
16 3 actionable tasks: 3 up-to-date
```

SpotBugs Report

Project Information

Project: MagicItems (spotbugsMain)

SpotBugs version: 4.9.6

Code analyzed:

- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\config\SecurityConfig.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\config\UserDetailsConfig.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\controller\AuthController.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\controller\MagicItemController.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\dto\auth\AuthRequestDto.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\dto\auth\AuthResponseDto.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\dto\data\MagicItemRequestDto.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\dto\data\MagicItemResponseDto\$MagicItemResponseDtoBuilder.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\dto\data\MagicItemResponseDto.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\dto\ErrorResponseDto\$ErrorResponseDtoBuilder.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\dto\ErrorResponseDto.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\entity\MagicItem\$MagicItemBuilder.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\entity\MagicItem.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\entity\Rarity.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\entity\UniversalNature.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\exception\GlobalExceptionHandler.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\exception\MagicItemNotFoundException.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\MagicItemsApplication.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\mapper\MagicItemMapper.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\repository\MagicItemRepository.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\security\JwtFilter.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\security\JwtUtils.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\security\package-info.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\service\AuthService.class
- C:\ITMO Software Engineering\Information Security\Lab1\MagicItems\build\classes\java\main\ru\itmo\cs\dandadan\magicitems\service\MagicItemService.class

Metrics

473 lines of code analyzed, in 25 classes, in 12 packages.

Metric	Total	Density*
High Priority Warnings		0.00
Medium Priority Warnings		0.00
Total Warnings	0	0.00

(* Defects per Thousand lines of non-commenting source statements)

Contents

- [Details](#)

Summary

Warning Type	Number
Total	0

Warnings

Click on a warning row to see full context information.

Details

Отчёты на OWASP:

▼ ✓ Run OWASP Dependency-Check

```
1 ▶ Run ./gradlew dependencyCheckAnalyze --console=plain
9
10 > Task :dependencyCheckAnalyze
11 Verifying dependencies for project MagicItems
12 Checking for updates and analyzing dependencies for vulnerabilities
13 Generating report for project MagicItems
14 Found 0 vulnerabilities in project MagicItems
15 Region [NODEAUDIT] : Not alive and dispose was called, filename: NODEAUDIT
16 Region [CENTRAL] : Not alive and dispose was called, filename: CENTRAL
17 Region [POM] : Not alive and dispose was called, filename: POM
20 [Incubating] Problems report is available at: file:///home/runner/work/MagicItems/MagicItems/build/reports/problems/problems-report.html
21 Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.
22 You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your own scripts or plugins.
23 For more on this, please refer to https://docs.gradle.org/8.14/userguide/command_line_interface.html#sec:command_line_warnings in the Gradle documentation.
24 BUILD SUCCESSFUL in 1m 36s
25 1 actionable task: 1 executed
```



DEPENDENCY-CHECK

Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies. False positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS-IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

[How to read the report](#) | [Suppressing false positives](#) | [Getting Help: github issues](#)

🔗 [Sponsor](#)

Project: root project 'MagicItems'

ru.lmo.cs.dandadan:MagicItems:1.0

Scan Information ([show all](#)):

- dependency-check version: 12.1.6
- Report Generated On: Wed, 15 Oct 2025 00:46:24 +0300
- Dependencies Scanned: 68 (68 unique)
- Vulnerable Dependencies: 0
- Vulnerabilities Found: 0
- Vulnerabilities Suppressed: 0

Summary

Summary of Vulnerable Dependencies([click to show all](#))

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
------------	-------------------	---------	------------------	-----------	------------	----------------

Dependencies (vulnerable)

This report contains data retrieved from the [National Vulnerability Database](#).
This report may contain data retrieved from the [USCA Known Exploited Vulnerability Catalog](#).
This report may contain data retrieved from the [Github Advisory Database \(via NPM Audit API\)](#).
This report may contain data retrieved from [RetpauJS](#).
This report may contain data retrieved from the [Sonatype OSS Index](#).

Ссылка на последнего успешного запуска pipeline:

<https://github.com/17starplatinum/MagicItems/actions/runs/18520656889/job/52779677145>